
Monitoramento de ransomware em nível de kernel

Detecção em tempo real com eBPF e análise de entropia de Shannon.

ALUNO

Kauã Junior Silva Soares

RA

231024061

ORIENTADOR

Prof. Dr. Kelton A. Pontara
da Costa

INSTITUIÇÃO

UNESP Bauru — Ciência da
Computação

Limitações da detecção por assinatura

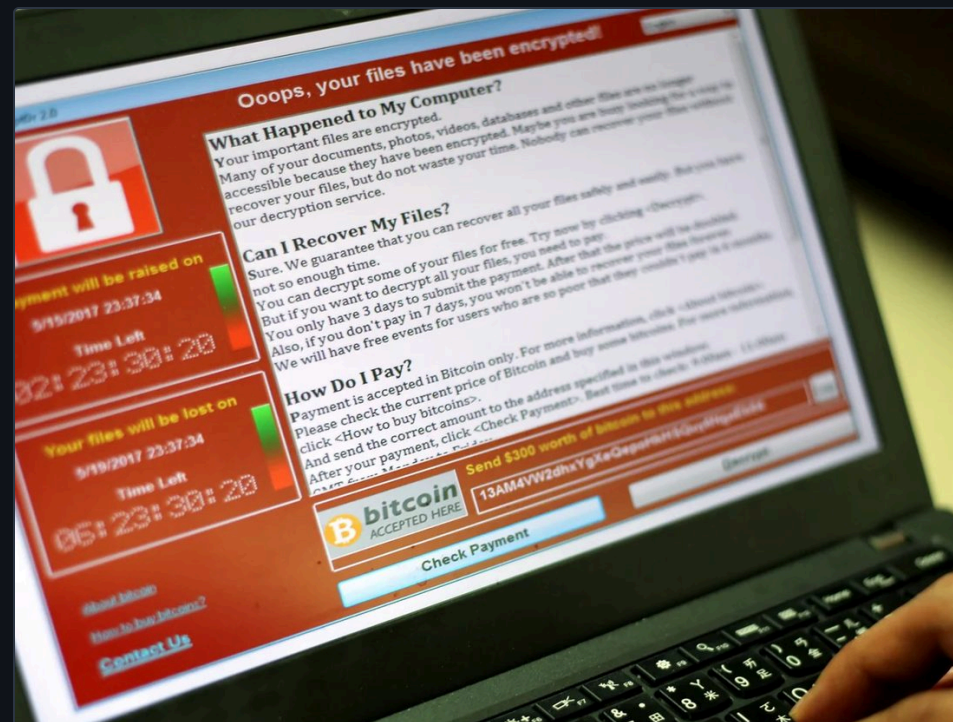
Antivírus tradicional compara o binário com uma lista de assinaturas conhecidas. Toda variante nova de ransomware nasce fora dessa lista.

Assinatura é reativa

Só reconhece o que já foi catalogado. Recompilar o malware muda a assinatura e derruba a detecção.

Comportamento não muda

Qualquer ransomware precisa fazer a mesma coisa: ler muitos arquivos e sobrescrever com dado cifrado. Isso dá pra observar.



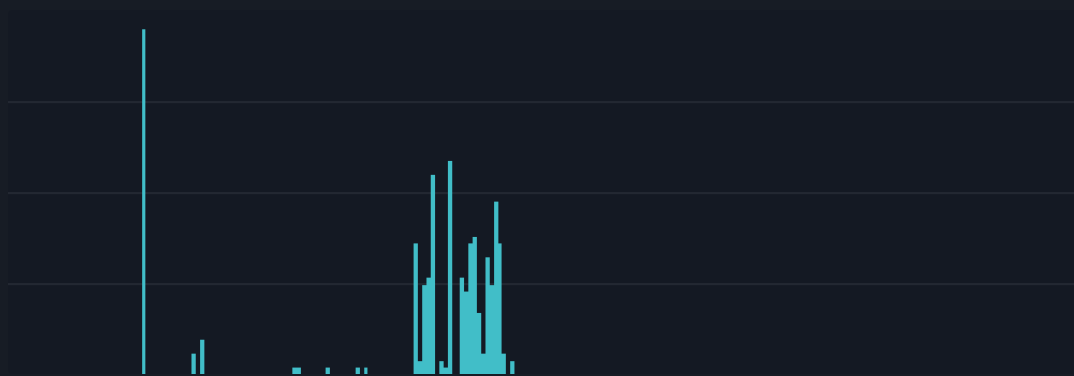
WannaCry, 2017. Mais de 200 mil máquinas em 150 países.

Entropia de Shannon como indicador

A entropia de Shannon mede quão imprevisível é o próximo byte, de 0 a 8 bits. Texto tem estrutura e fica baixo. Cifra bem feita é indistinguível de aleatório e encosta no teto.

TEXTO COMUM

4,13 bits



Distribuição dos 256 valores de byte. Só a faixa ASCII aparece.

DADO CIFRADO

7,98 bits



Todos os 256 valores igualmente prováveis. Distribuição chapada.

Caracterização do comportamento de escrita

Escrevi dois simuladores. Um gera carga benigna de texto, o outro sobrescreve com bytes de /dev/urandom, imitando a cifragem em massa.

```
# antes: texto legível
```

```
$ head -c 32 benigno_0000.txt  
Lorem ipsum dolor sit amet, cons
```

```
# depois do sim_encrypt
```

```
$ head -c 32 benigno_0000.txt | od -An -tx1  
9b a9 ab 5e e1 43 ab e1 ea ae 19  
5b 45 37 a3 65 7c 0f d2 88 41 b3
```

```
jiraiya23@ios-sys-webview:~/Área de Trabalho/estudos/tcc/ebpf-ransomware-detector$ ./tests/sim_benign /tmp/demo --files 5 --size  
-kb 8  
[*] carga benigna: 5 arquivos de 8 KB em /tmp/demo  
[*] carga benigna concluída: 5 arquivos  
jiraiya23@ios-sys-webview:~/Área de Trabalho/estudos/tcc/ebpf-ransomware-detector$ head -c 30 /tmp/demo/benigno_0000.txt && echo  
Lorem ipsum dolor sit amet, co  
jiraiya23@ios-sys-webview:~/Área de Trabalho/estudos/tcc/ebpf-ransomware-detector$ ./tests/sim_encrypt /tmp/demo  
[*] simulando cifragem em /tmp/demo, max 1000 arquivos  
[CIFRADO] /tmp/demo/benigno_0004.txt (8192 bytes)  
[CIFRADO] /tmp/demo/benigno_0002.txt (8192 bytes)  
[CIFRADO] /tmp/demo/benigno_0003.txt (8192 bytes)  
[CIFRADO] /tmp/demo/benigno_0000.txt (8192 bytes)  
[CIFRADO] /tmp/demo/benigno_0001.txt (8192 bytes)  
[*] concluído: 5 arquivos em 24 ms (208.3 arq/s)  
jiraiya23@ios-sys-webview:~/Área de Trabalho/estudos/tcc/ebpf-ransomware-detector$ head -c 30 /tmp/demo/benigno_0000.txt && echo  
@e08@e?e4ee.e0e0he.e=ee
```

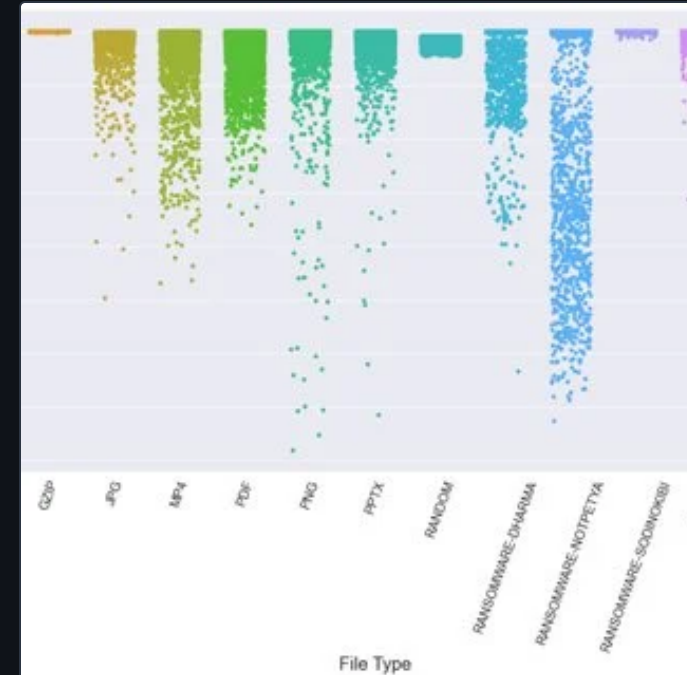
5 arquivos reescritos em 24 ms. O mesmo arquivo, antes e depois.

Ambiguidade da entropia como critério único

Arquivo .zip, .jpg e .mp4 também têm entropia perto de 8. Compactar uma pasta dispararia o alarme.

Custo do falso positivo

Matar o processo errado é pior que não detectar. Davies et al. (2020) mostram exatamente esse limite.



GZIP, JPG, MP4 e PNG ocupam a mesma faixa de entropia que Dharma, NotPetya e Sodinokibi.

Critérios de decisão

O bloqueio só dispara quando as três batem juntas. É essa combinação que separa ransomware de um simples zip -r.

CRITÉRIO 1

$H \geq 7,8$

Entropia por byte acima do limiar. Indica conteúdo cifrado.

CRITÉRIO 2

$\geq 64 \text{ KB}$

Volume total escrito. Filtra ruído e escrita pontual.

CRITÉRIO 3

≥ 10

Arquivos distintos tocados. Compactador escreve num arquivo só; ransomware espalha.

O terceiro critério é o que derruba o falso positivo do slide anterior.

Justificativa da escolha do eBPF

eBPF roda código dentro do kernel, em sandbox, sem compilar módulo e sem reiniciar a máquina.

Cobertura ampla

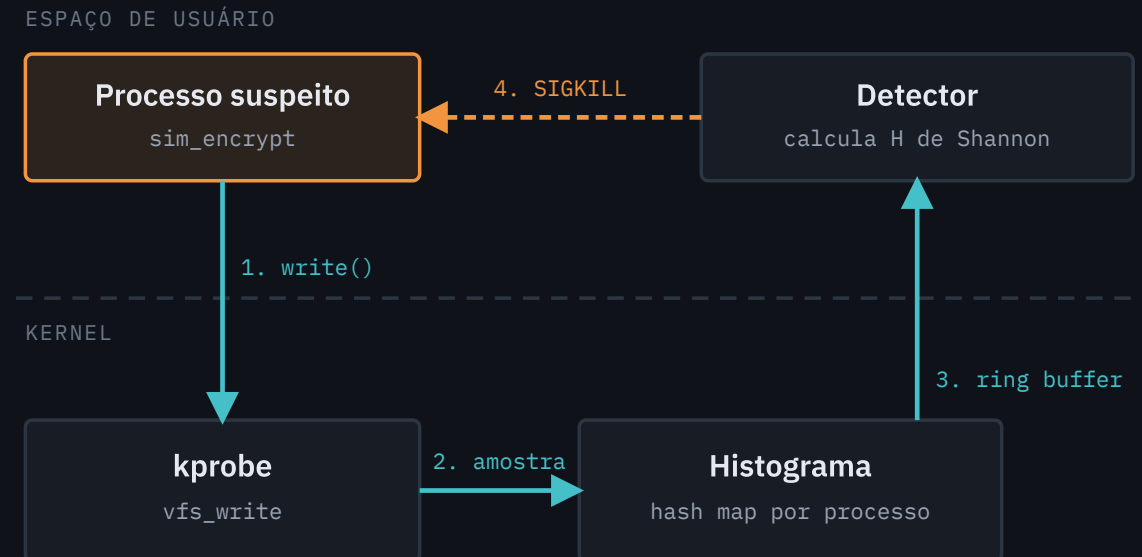
Um gancho em `vfs_write` pega toda escrita em disco, não importa se veio de `write`, `pwrite` ou outra syscall.

Seguro por construção

O verificador do kernel recusa o programa se ele puder travar o sistema. Não dá pra causar panic.

CO-RE

Compile uma vez, roda em versões diferentes de kernel sem recompilar.



Sem ponto flutuante no kernel:
o kernel só conta bytes, quem divide é o userspace.

Arquitetura do agente

```
// kernel: intercepta e acumula
SEC("kprobe/vfs_write")
int BPF_KPROBE(kprobe_vfs_write, ...)
{
    // lê 512 bytes da escrita
    bpf_probe_read_user(amostra, 512, buf);

    // conta frequência de cada byte
    for (i = 0; i < 512; i++)
        st->histograma[amostra[i]]++;

    // 3 critérios batem? avisa o userspace
    bpf_ringbuf_submit(ev, 0);
}
```

1. Coleta no kernel

O histograma dos 256 valores de byte é acumulado por processo num hash map.

2. Transporte via ring buffer

Quando os critérios batem, o evento sai do kernel pro espaço de usuário.

3. Decisão no espaço de usuário

A entropia é calculada aqui, porque o verificador não permite ponto flutuante dentro do kernel.

Restrições do verificador eBPF

O kernel só aceita o programa se conseguir provar que ele é seguro. Três paredes que bati:

Ausência de ponto flutuante

Por isso o cálculo da entropia mudou de lado e foi pro espaço de usuário.

Stack de 512 bytes

Minha struct tem 1312. A solução foi guardar uma cópia zerada num mapa e copiar de lá.

Chamada a memset não suportada

O compilador transformava meu laço de zerar array em memset, que o alvo BPF não suporta. Resolvido com `-fno-builtin`.

```
jiraiya23@ios-sys-webview:~/Área de Trabalho/estudos/tcc/ebpf-ransomware-detector$ clang-18 -g -O2 -target bpf -D__TARGET_ARCH_x86 -I./include -I/usr/include/x86_64-linux-gnu -c src/bpf/ransomware_detect.bpf.c -o /tmp/erro.o
src/bpf/ransomware_detect.bpf.c:147:27: error: A call to built-in function 'memset' is not supported.
147 |         st->histograma[i] = 0;
    |                             ^
1 error generated.
```

Erro real, na linha 147. Um laço trivial de zerar array virou memset e o alvo BPF recusou.

Resultados preliminares

```
o limiar
[verbose] PID 365589 (sim_benign): H=4.1300 bits, abaixo d
o limiar
[verbose] PID 365589 (sim_benign): H=4.1300 bits, abaixo d
o limiar
[verbose] PID 365589 (sim_benign): H=4.1300 bits, abaixo d
o limiar
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9387 bits/byte BY
TES=163840 TEMPO=81833.4 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9363 bits/byte BY
TES=163840 TEMPO=81840.0 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9295 bits/byte BY
TES=163840 TEMPO=81846.3 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9401 bits/byte BY
TES=163840 TEMPO=81851.6 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9280 bits/byte BY
TES=163840 TEMPO=81858.2 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9142 bits/byte BY
TES=163840 TEMPO=81863.6 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9289 bits/byte BY
TES=163840 TEMPO=81869.2 ms
[ALERTA] PID=365642 PROC=sim_encrypt H=7.9279 bits/byte BY
TES=163840 TEMPO=81874.8 ms
jiraiya23@ios-sys-webview:~/Area de Trabalho/estudos/tc
r$ ./tests/sim_encrypt /tmp/demo
[*] simulando cifragem em /tmp/demo, max 1000 arquivos
[CIFRADO] /tmp/demo/benigno_0005.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0023.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0011.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0035.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0019.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0006.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0021.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0036.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0030.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0032.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0016.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0015.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0025.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0028.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0018.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0008.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0004.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0039.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0029.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0031.txt (32768 bytes)
[CIFRADO] /tmp/demo/benigno_0002.txt (32768 bytes)
```

Esquerda: o detector. Direita: o simulador cifrando.

SIM_BENIGN

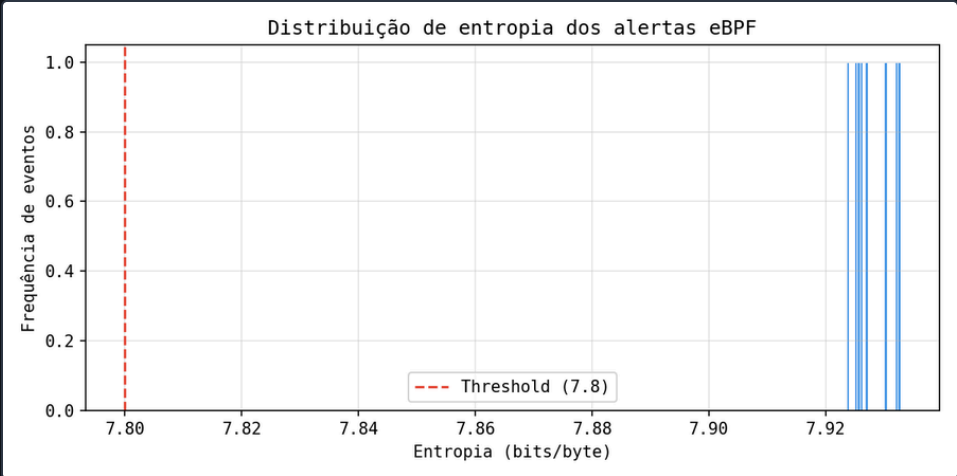
4,13

bits/byte. Abaixo
do limiar,
ignorado.

SIM_ENCRYPT

7,93

bits/byte. Alerta
disparado.



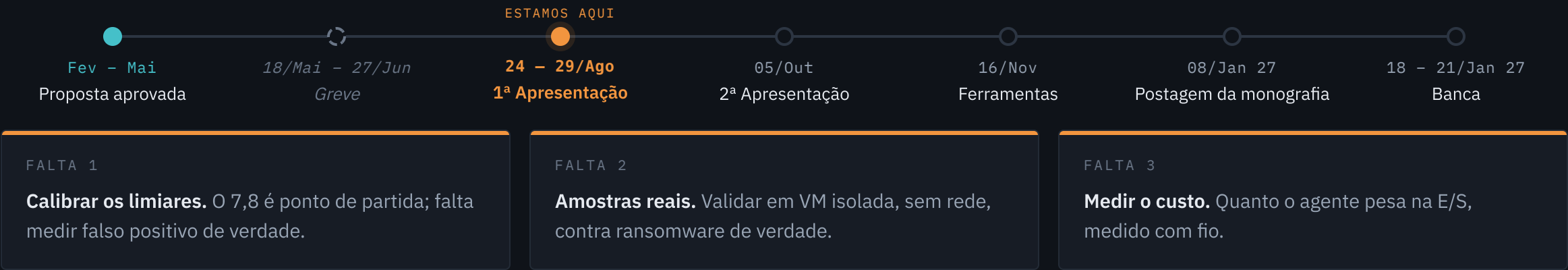
8 alertas registrados, todos entre 7,92 e 7,93.

Comparação com abordagens existentes

ABORDAGEM	ONDE RODA	BLOQUEIA?	VÊ O CONTEÚDO?
auditd	Kernel	Não, só registra	Não
inotify	Userspace	Não, avisa depois	Não
Este trabalho	Kernel (eBPF)	Sim, SIGKILL	Sim, entropia

A diferença é chegar antes: o inotify avisa depois que o arquivo já fechou. O objetivo aqui é matar o processo antes de o diretório inteiro ser cifrado.

Cronograma e trabalhos futuros



Obrigado

Perguntas?

CONTATO

Kauã Junior Silva Soares

ORIENTADOR

Prof. Dr. Kelton A. Pontara da Costa

CÓDIGO

ebpf-ransomware-detector